

Name : Narendra Kumar S

Date : 13-08-2025

Task :

1 create a dependency injection sample for painter class

1.1 inject tools (or paintbrush) to painter object via dependency injection

Solution:

Painter:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

```
using System;
```

```
namespace Day8Deplnj
{
    public class Painter
    {
        public string Name { get; }
        public ITool Tool { get; }

        public Painter(string name, ITool tool)
        {
            Name = name;
            Tool = tool;
        }

        public string Paint(string surface)
        {
            return $"{Name} is painting {surface} using {Tool.Name} ({Tool.Width} cm)";
        }
    }
}
```

InterFace

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
```

```
namespace Day8Deplnj
{
    public interface ITool
    {
        string Name { get; }
        double Width { get; }
        bool IsForFineWork();
    }
}
```

PaintBrush

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Runtime.ConstrainedExecution;
using System.Text;
using System.Threading.Tasks;
```

```
namespace Day8Deplnj
{
    public class PaintBrush : ITool
    {
        public PaintBrush(string name, double width)
        {
            Name = name;
            Width = width;
        }

        public string Name { get; }
        public double Width { get; }

        public bool IsForFineWork()
        {
            return Width <= 1.0;
        }
    }
}
```

```
}  
}
```

SprayGun:

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
  
namespace Day8DeplnJ  
{  
    public class SprayGun : ITool  
    {  
        public SprayGun(string name, double width)  
        {  
            Name = name;  
            Width = width;  
        }  
  
        public string Name { get; }  
        public double Width { get; }  
  
        public bool IsForFineWork()  
        {  
            return false;  
        }  
    }  
}
```

Program

```
using System;  
using Day8DeplnJ;  
  
class Program  
{  
    static void Main(string[] args)  
    {  
        ITool brush = new PaintBrush("Fine Tip Brush", 0.5);  
        ITool gun = new SprayGun("Pro Spray Gun", 5.0);  
    }  
}
```

```

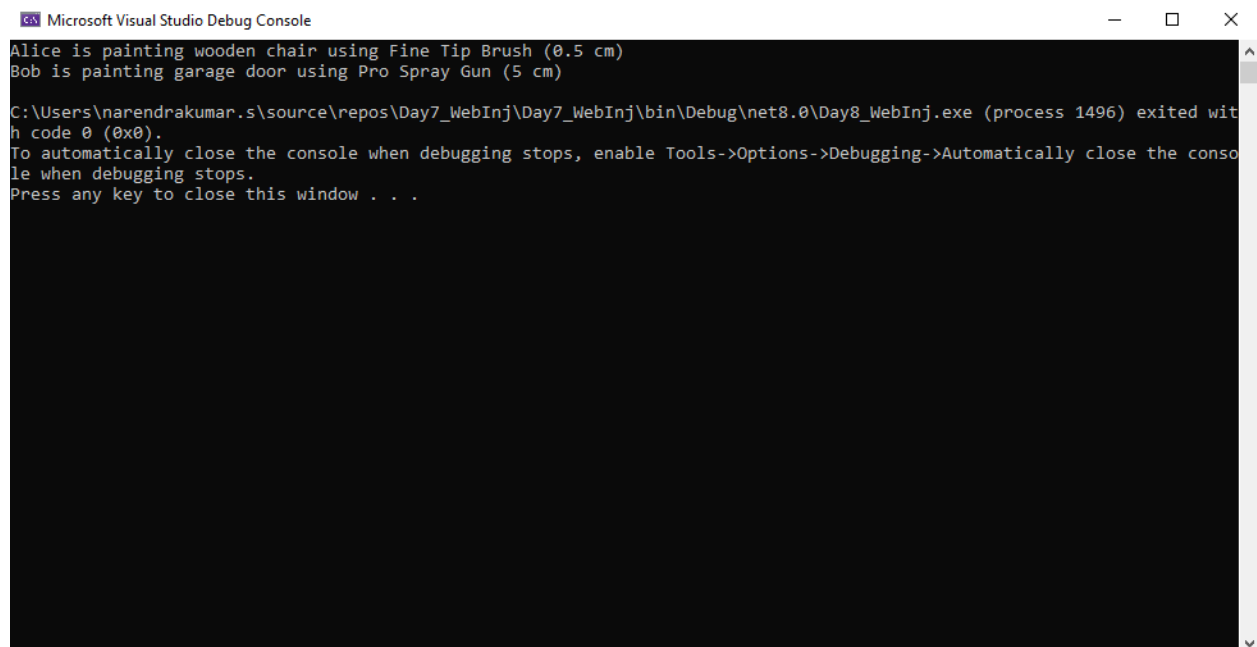
Painter painter1 = new Painter("Alice", brush);
Painter painter2 = new Painter("Bob", gun);

Console.WriteLine(painter1.Paint("wooden chair"));
Console.WriteLine(painter2.Paint("garage door"));

}
}

```

Output:



```

Microsoft Visual Studio Debug Console
Alice is painting wooden chair using Fine Tip Brush (0.5 cm)
Bob is painting garage door using Pro Spray Gun (5 cm)
C:\Users\narendrakumar.s\source\repos\Day7_WebInj\Day7_WebInj\bin\Debug\net8.0\Day8_WebInj.exe (process 1496) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .

```

2.create the atleast 10 Nunit test case

2.1 run the test both using visual studio and command line

Solution:

```

using Day8Deplnj;
using Moq;
using NUnit.Framework;

```

namespace PainterTest

```

{
    public class PainterTests
    {
        Painter _sut;
        Mock<ITool> mockTool;

        [SetUp]
        public void Setup()
        {
            mockTool = new Mock<ITool>();
            mockTool.Setup(t => t.Name).Returns("Fine Tip Brush");
            mockTool.Setup(t => t.Width).Returns(0.5);
            mockTool.Setup(t => t.IsForFineWork()).Returns(true);

            _sut = new Painter("Alex", mockTool.Object);
        }

        [TestCase("Fine Tip Brush", 0.5)]
        [TestCase("Medium Brush", 1.0)]
        [TestCase("Detail Brush", 0.2)]
        public void IsSmallWidth(string name, double width)
        {
            mockTool.Setup(t => t.Name).Returns(name);
            mockTool.Setup(t => t.Width).Returns(width);
            mockTool.Setup(t => t.IsForFineWork()).Returns(true);

            _sut = new Painter("Alex", mockTool.Object);

            Assert.That(_sut.Tool.IsForFineWork(), Is.True,
                $"Tool '{name}' with width {width} should be for fine work");
        }

        [TestCase("Wide Brush", 2.0)]
        [TestCase("Spray Gun", 5.0)]
        public void IsLargeWidth(string name, double width)
        {
            mockTool.Setup(t => t.Name).Returns(name);
            mockTool.Setup(t => t.Width).Returns(width);
            mockTool.Setup(t => t.IsForFineWork()).Returns(false);

            _sut = new Painter("Alex", mockTool.Object);

            Assert.That(_sut.Tool.IsForFineWork(), Is.False,
                $"Tool '{name}' with width {width} should not be for fine work");
        }
    }
}

```

```

}

[TestCase("Alex")]
[TestCase("Bob")]
public void IsPainterNameMatch(string painterName)
{
    _sut = new Painter(painterName, mockTool.Object);

    Assert.That(_sut.Name, Is.EqualTo(painterName), "Painter name did not match");
}

[TestCase("Wall")]
[TestCase("Door")]
[TestCase("Chair")]
public void Method_Return_Check(string surface)
{
    mockTool.Setup(t => t.Name).Returns("Any Tool");
    _sut = new Painter("Alex", mockTool.Object);

    var result = _sut.Paint(surface);

    StringAssert.Contains($"painting {surface}", result,
        "Paint method did not include correct surface name");
}

[TestCase("Air Spray Gun", 6.0)]
[TestCase("Pressure Gun", 4.5)]
public void SprayGun_Check(string name, double width)
{
    var sprayGunMock = new Mock<ITool>();
    sprayGunMock.Setup(t => t.Name).Returns(name);
    sprayGunMock.Setup(t => t.Width).Returns(width);
    sprayGunMock.Setup(t => t.IsForFineWork()).Returns(false);

    Assert.That(sprayGunMock.Object.IsForFineWork(), Is.False,
        $"SprayGun '{name}' should never be fine work");
}

[TestCase("Brush", 0.5)]
[TestCase("Brush", 2.0)]
public void IsPainterToolNameMatch(string toolName, double width)
{
    mockTool.Setup(t => t.Name).Returns(toolName);
    mockTool.Setup(t => t.Width).Returns(width);
}

```

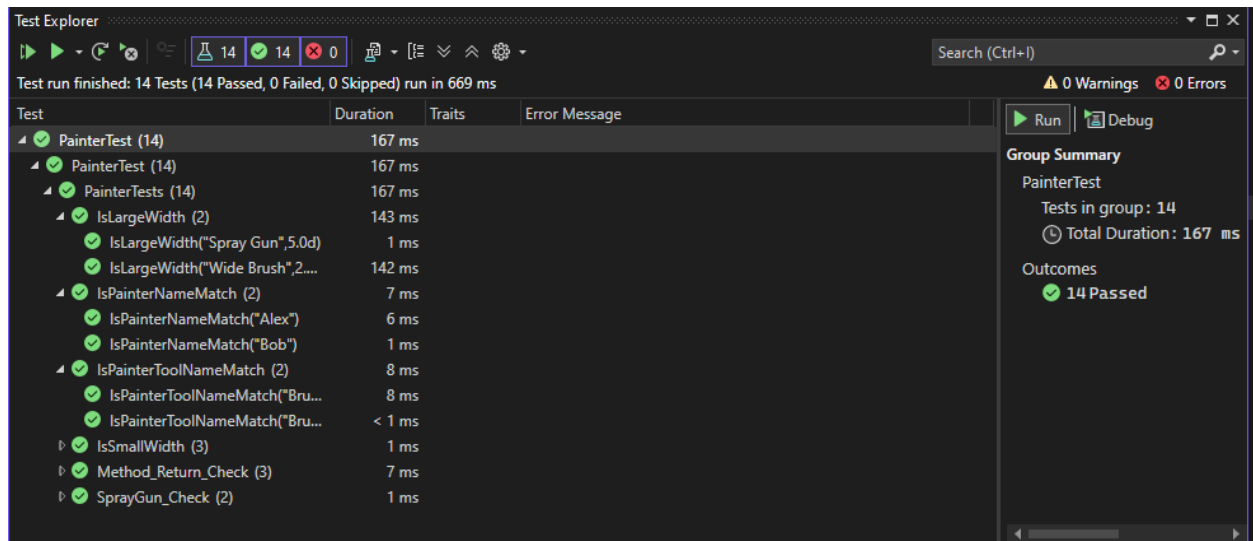
```

        _sut = new Painter("Alex", mockTool.Object);

        Assert.That(_sut.Tool.Name, Is.EqualTo(toolName),
            "Painter's tool name did not match expected");
    }
}
}

```

Output Using the Visual Studio :



Test run with the Command Line Interface :

Solution :

```

C:\Users\narendrakumar.s\source\repos\Day7_WebInj\PainterTest>dotnet test
PainterTest.csproj

```

Output:

```
Select C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.6159]
(c) Microsoft Corporation. All rights reserved.

C:\Users\narendrakumar.s\source\repos\Day7_WebInj\PainterTest>dotnet test PainterTest.csproj
Restore complete (0.9s)
  Day8_WebInj succeeded (4.8s) → C:\Users\narendrakumar.s\source\repos\Day7_WebInj\Day7_WebInj\bin\Debug\net8.0\Day8_WebInj.dll
  PainterTest succeeded (1.3s) → bin\Debug\net8.0\PainterTest.dll
NUnit Adapter 4.5.0.0: Test execution started
Running all tests in C:\Users\narendrakumar.s\source\repos\Day7_WebInj\PainterTest\bin\Debug\net8.0\PainterTest.dll
  NUnit3TestExecutor discovered 14 of 14 NUnit test cases using Current Discovery mode, Non-Explicit run
NUnit Adapter 4.5.0.0: Test execution complete
  PainterTest test succeeded (3.2s)

Test summary: total: 14, failed: 0, succeeded: 14, skipped: 0, duration: 3.1s
Build succeeded in 11.2s

C:\Users\narendrakumar.s\source\repos\Day7_WebInj\PainterTest>
```